Persistența în baza de date (JDBC)

MAP – Cursul 6 · PostgreSQL · pachetul `repository.db`

JDBC: API + driver

- **JDBC** = interfețe din `java.sql` (`Connection`, `PreparedStatement`, `ResultSet`). Codul nostru depinde doar de ele.
- **Driverul** (`org.postgresql:postgresql`, din Gradle) le implementează pentru PostgreSQL. Altă bază de date = alt driver, același cod Java.
- URL-ul spune unde ne conectăm: `jdbc:postgresql://localhost:5432/socialnetwork` (host, port, baza de date).

Pașii unei operații

```
String sql = "SELECT * FROM users WHERE id = ?";
try (Connection c = DriverManager.getConnection(url, user, password);   // 1. conexiunea
     PreparedStatement ps = c.prepareStatement(sql)) {                  // 2. instrucțiunea
    ps.setLong(1, id);                                                  // 3. parametrii (de la 1)
    try (ResultSet rs = ps.executeQuery()) {                            // 4. execuția
        while (rs.next())                                               // 5. rând cu rând
            System.out.println(rs.getString("first_name"));
    }
} catch (SQLException e) { ... }                // 6. resursele se închid automat (try-with-resources)
```

Interfețele principale

Interfață	Rol
`Connection`	sesiunea cu baza de date; se închide mereu
`PreparedStatement`	instrucțiune SQL cu parametri `?`
`ResultSet`	cursor peste rândurile rezultate; `next()` avansează
`SQLException`	excepție **checked** aruncată de aproape orice metodă JDBC

Execuția

Metodă	Pentru	Întoarce
`executeQuery()`	SELECT	`ResultSet`
`executeUpdate()`	INSERT, UPDATE, DELETE	`int`: rânduri modificate

- La UPDATE și DELETE, 0 rânduri modificate = id-ul nu există.

PreparedStatement, nu concatenare

```
// GREȘIT: SQL injection
"... WHERE last_name = '" + nume + "'"
// nume = "x' OR '1'='1"  ->  toate rândurile
```

```
// CORECT: valoarea e trimisă separat
"... WHERE last_name = ?"
ps.setString(1, nume);   // nu devine cod SQL
```

Metodele repository-ului

Metodă	SQL	Rezultat
`findAll()`	`SELECT * FROM users`	fiecare rând → un `Utilizator`
`findOne(id)`	`SELECT ... WHERE id = ?`	`Optional.of(u)` sau `Optional.empty()`
`save(u)`	`INSERT INTO users ...`	după validare; `Optional.of(u)` dacă id-ul există deja
`update(u)`	`UPDATE ... WHERE id = ?`	0 rânduri modificate → `Optional.of(u)`
`delete(id)`	`DELETE ... WHERE id = ?`	utilizatorul șters (găsit întâi cu `findOne`)

Repository și Factory (principii OOP)

- `SQLException` se împachetează în `RepositoryException` (unchecked): interfața `Repository` nu depinde de tipul stocării.
- Aceeași interfață pentru memorie, fișier și bază de date (**DIP**); `RepositoryFactory` alege implementarea din `config.properties`.
- Am adăugat stocarea în baza de date fără să modificăm `Main` sau `Repository` (**Open/Closed**).
- `stream()` și `findAll(Predicate)` merg și pe baza de date, dar filtrează **în memorie**. Pentru tabele mari, filtrarea se face în SQL (`WHERE`).

Conexiunea, instrucțiunea și `ResultSet`-ul se deschid în `try-with-resources`. O conexiune neînchisă rămâne ocupată pe server. Datele venite de la utilizator ajung în SQL doar prin parametri `?`.

Codul complet al operațiilor (UtilizatorDbRepository)

Câmpuri: `url`, `username`, `password`, `validator`, primite prin constructor. Fiecare metodă urmează pașii de mai sus.

findAll

```
@Override
public Iterable<Utilizator> findAll() {
    List<Utilizator> users = new ArrayList<>();
    try (Connection connection = DriverManager.getConnection(url, username, password);
         PreparedStatement statement = connection.prepareStatement("SELECT * FROM users ORDER BY id");
         ResultSet resultSet = statement.executeQuery()) {

        while (resultSet.next()) {
            Long id = resultSet.getLong("id");
            String firstName = resultSet.getString("first_name");
            String lastName = resultSet.getString("last_name");

            Utilizator utilizator = new Utilizator(firstName, lastName);
            utilizator.setId(id);
            users.add(utilizator);
        }
    } catch (SQLException e) {
        throw new RepositoryException("Eroare la findAll: " + e.getMessage(), e);
    }
    return users;
}
```

findOne

```
@Override
public Optional<Utilizator> findOne(Long id) {
    if (id == null)
        throw new IllegalArgumentException("id must be not null");
    try (Connection connection = DriverManager.getConnection(url, username, password);
         PreparedStatement statement = connection.prepareStatement("SELECT * FROM users WHERE id = ?")) {

        statement.setLong(1, id);
        try (ResultSet resultSet = statement.executeQuery()) {
            if (resultSet.next()) {
                Utilizator utilizator = new Utilizator(resultSet.getString("first_name"),
                        resultSet.getString("last_name"));
                utilizator.setId(id);
                return Optional.of(utilizator);
            }
            return Optional.empty();
        }
    } catch (SQLException e) {
        throw new RepositoryException("Eroare la findOne: " + e.getMessage(), e);
    }
}
```

save

```
@Override
public Optional<Utilizator> save(Utilizator entity) {
    if (entity == null)
        throw new IllegalArgumentException("entity must be not null");
    validator.validate(entity);
    if (findOne(entity.getId()).isPresent())
        return Optional.of(entity);

    String sql = "INSERT INTO users (id, first_name, last_name) VALUES (?, ?, ?)";
    try (Connection connection = DriverManager.getConnection(url, username, password);
         PreparedStatement ps = connection.prepareStatement(sql)) {

        ps.setLong(1, entity.getId());
        ps.setString(2, entity.getFirstName());
        ps.setString(3, entity.getLastName());
        ps.executeUpdate();
    } catch (SQLException e) {
        throw new RepositoryException("Eroare la save: " + e.getMessage(), e);
    }
    return Optional.empty();
}
```

update

```
@Override
public Optional<Utilizator> update(Utilizator entity) {
    if (entity == null)
        throw new IllegalArgumentException("entity must be not null!");
    validator.validate(entity);

    String sql = "UPDATE users SET first_name = ?, last_name = ? WHERE id = ?";
    try (Connection connection = DriverManager.getConnection(url, username, password);
         PreparedStatement ps = connection.prepareStatement(sql)) {

        ps.setString(1, entity.getFirstName());
        ps.setString(2, entity.getLastName());
        ps.setLong(3, entity.getId());
        int randuri = ps.executeUpdate();
        return randuri == 0 ? Optional.of(entity) : Optional.empty();
    } catch (SQLException e) {
        throw new RepositoryException("Eroare la update: " + e.getMessage(), e);
    }
}
```

delete

```
@Override
public Optional<Utilizator> delete(Long id) {
    Optional<Utilizator> gasit = findOne(id);
    if (gasit.isEmpty())
        return gasit;

    try (Connection connection = DriverManager.getConnection(url, username, password);
         PreparedStatement ps = connection.prepareStatement("DELETE FROM users WHERE id = ?")) {

        ps.setLong(1, id);
        ps.executeUpdate();
    } catch (SQLException e) {
        throw new RepositoryException("Eroare la delete: " + e.getMessage(), e);
    }
    return gasit;
}
```